



Elektronik & Programmering med Arduino

NYBÖRJARKURS • 5 TRÄFFAR

Fem träffar. Fem moduler. Ett fungerande larm.

KURSÖVERSIKT

Fem träffar. Fem moduler.



MODUL 01

LED & krets

Digital output, Ohms lag



MODUL 02

PWM & RGB

analogWrite, färgblandning



MODUL 03

Digital input

Knapp, buzzer, logik



MODUL 04

Analog input

Sensorer, Serial Monitor



MODUL 05

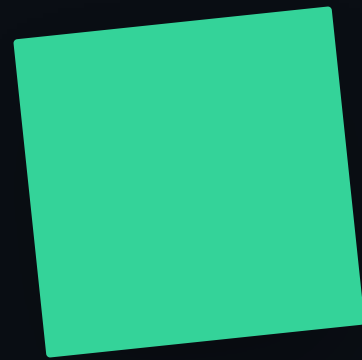
Integration

Hackathon: bygg larmet

Varje träff bygger vidare på den föregående.

INFÖR TRÄFF 1 • KLASSRUMSVKTYGET

Klassrummets viktigaste verktyg



GRÖN

Allt flyter på

+



RÖD

Jag behöver hjälp

TRÄFF 1 AV 5

LED & krets

Modul 1: **Digital output, Ohms lag**

D A G E N S U P P L Ä G G

Idag — två timmar.

00:00 — 00:15

Slutmålet & Arduinon

Vi tittar på det färdiga larmet och ramar in vad ett Arduino-kort egentligen är.

00:35 — 01:10

Bygg kretsen & ladda upp Blink

Praktiskt: LED + resistor på breadboarden, första uppladdningen från IDE:n.

00:15 — 00:35

Krets, spänning, ström

Kort genomgång av vad en krets är och varför LED:en behöver en resistor.

01:10 — 02:00

Din egen rytm & delresultat

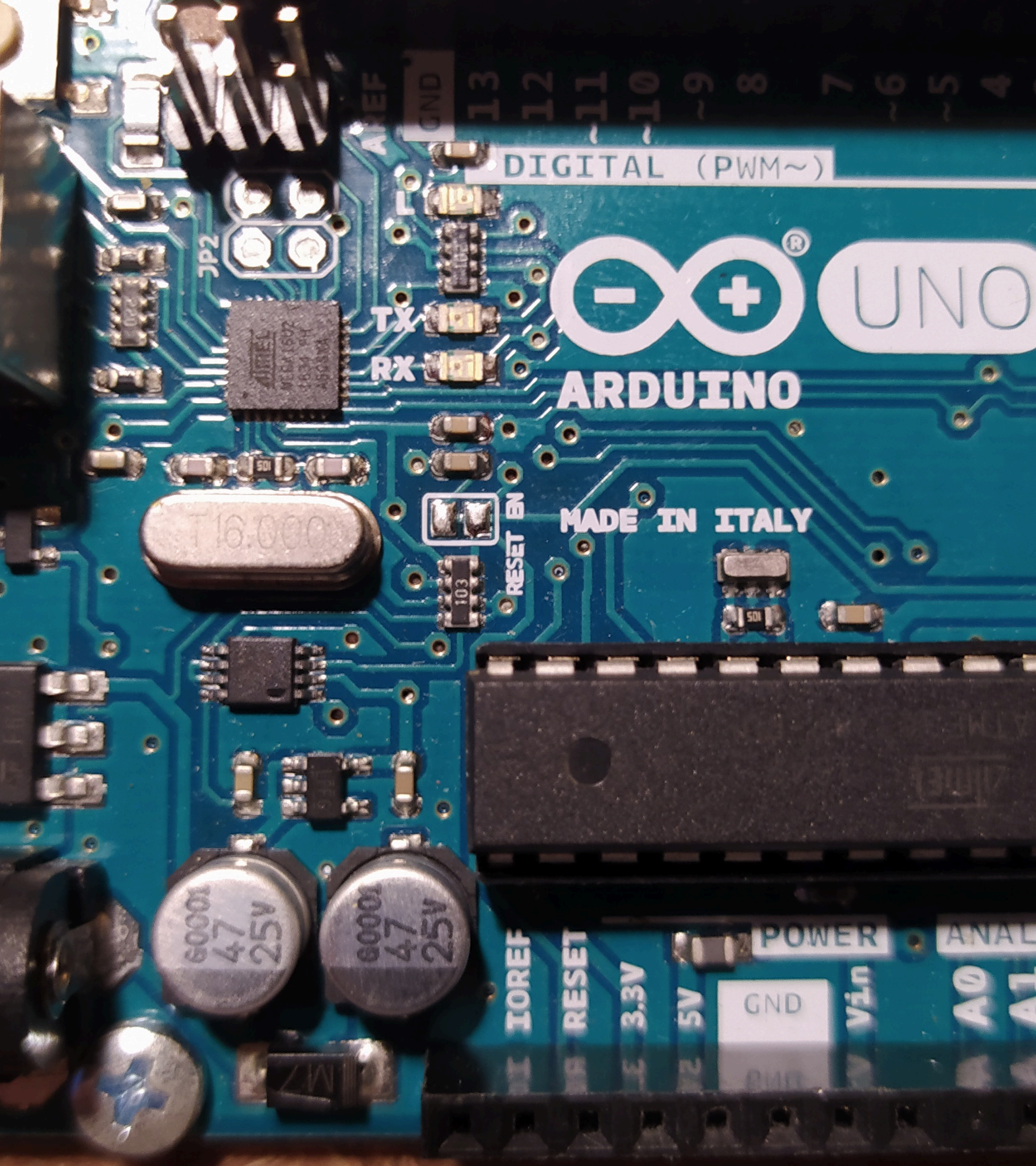
Fri övning: ändra delay-värdena till ett eget mönster. Alla visar upp.



SLUTMÅLET

Det här ska vi bygga.

Ett larm som känner av mörker, lyser
upp som stämningsljus och tjuder när
någon rör det.



ARDUINO UNO R3

Mikrokontrollern.

En **mikrokontroller** — processor, minne och in-/utgångar på ett chip.
Kör ett enda program, helt förutsägbart.

Arduino UNO R3 • ATmega328P
16 MHz • 32 KB flash • 2 KB SRAM

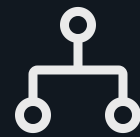
TRE GRUNDBEGREPP

Spänning, ström, GND.



SPÄNNING

Volt · V



STRÖM

Ampere · A

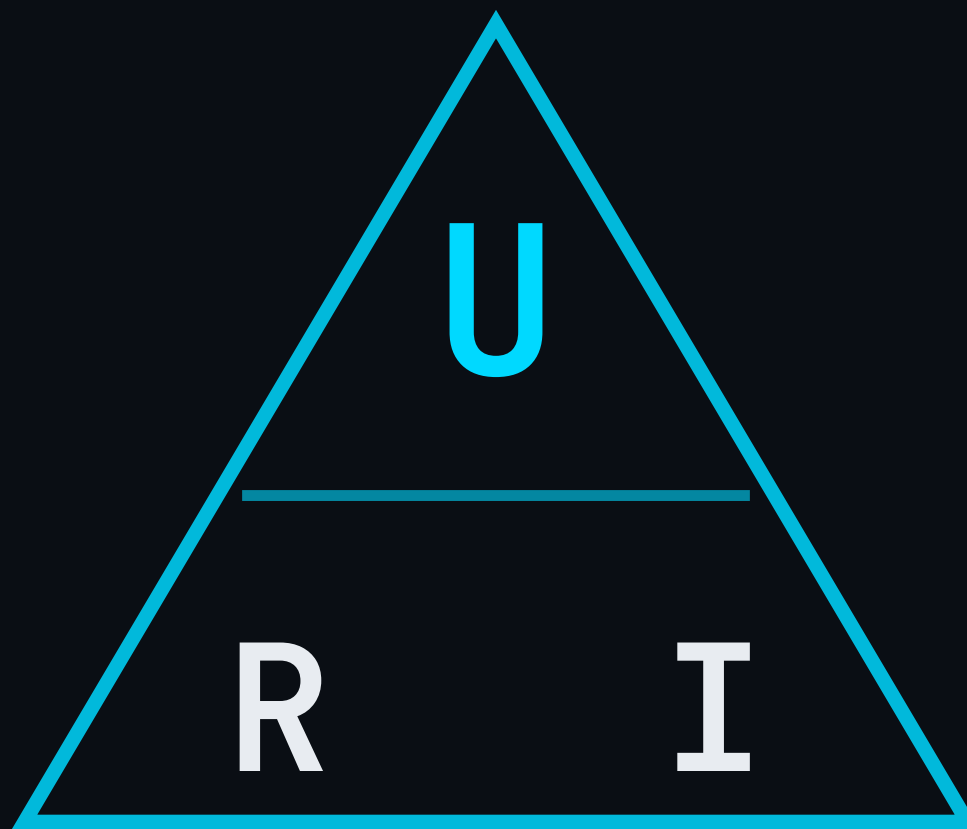


GND

0 V · referens

OHMS LAG

$$U = R \cdot I$$



täck variabeln ni söker

Räkneexempel
och
framspänningsfall:
→ Bilaga F i
kompendiet

K R E T S E N

Kretsen — som ett vattensystem.

Strömmen går ut från en **utgångspinne** (t.ex. pin 13 när den är HIGH), genom komponenterna, och tillbaka till **GND**.

Utan bromsning: **LED:en brinner upp**.

Lösningen: en **strypventil** — en resistor på $220\ \Omega$ i serie.

MINNESREGEL

Ingen krets utan väg tillbaka till GND. Strömmen måste kunna fullborda varvet — annars händer ingenting.

```
pin 13  →  LED (+)
LED (-)  →  220  $\Omega$ 
220  $\Omega$   →  GND
```

LED & resistor.

LED:en

- Har polaritet. Kör du den baklänges lyser den inte.
- Långt ben = anod (+) → till signalpinnen (pin 13).
- Kort ben + platt kant = katod (–) → till GND.
- Tål ca 20 mA. Mer → den brinner upp permanent.

Kör du den baklänges går den inte sönder — den lyser bara inte. Bara att vända.

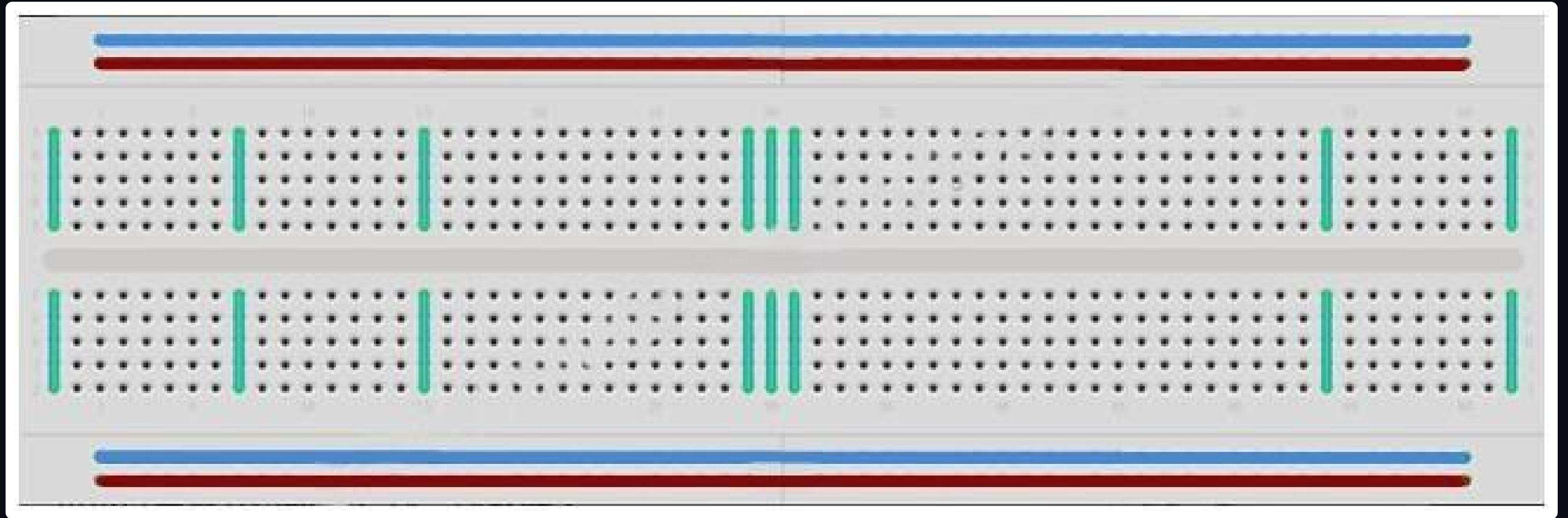
Resistorn

- Passiv komponent. Ingen polaritet — koppla hur du vill.
- Värdet läses på färgringarna.
- 220 Ω (4-band) = röd · röd · brun + guld.
- 220 Ω (5-band) = röd · röd · svart · svart + brun.



röd · röd · brun
2 · 2 · $\times 10^1$
= 220 Ω

Fem hål = en nod.



- Fem hål i rad = en **nod**
- Raden bredvid = **egen** nod

- Gapet i mitten **bryter**
- **+/-**-skenorna går hela vägen

FELSÖKNING • REGEL #1

Kontrollera raden först.

**90 % av alla nybörjarfel
är ett håll fel.**

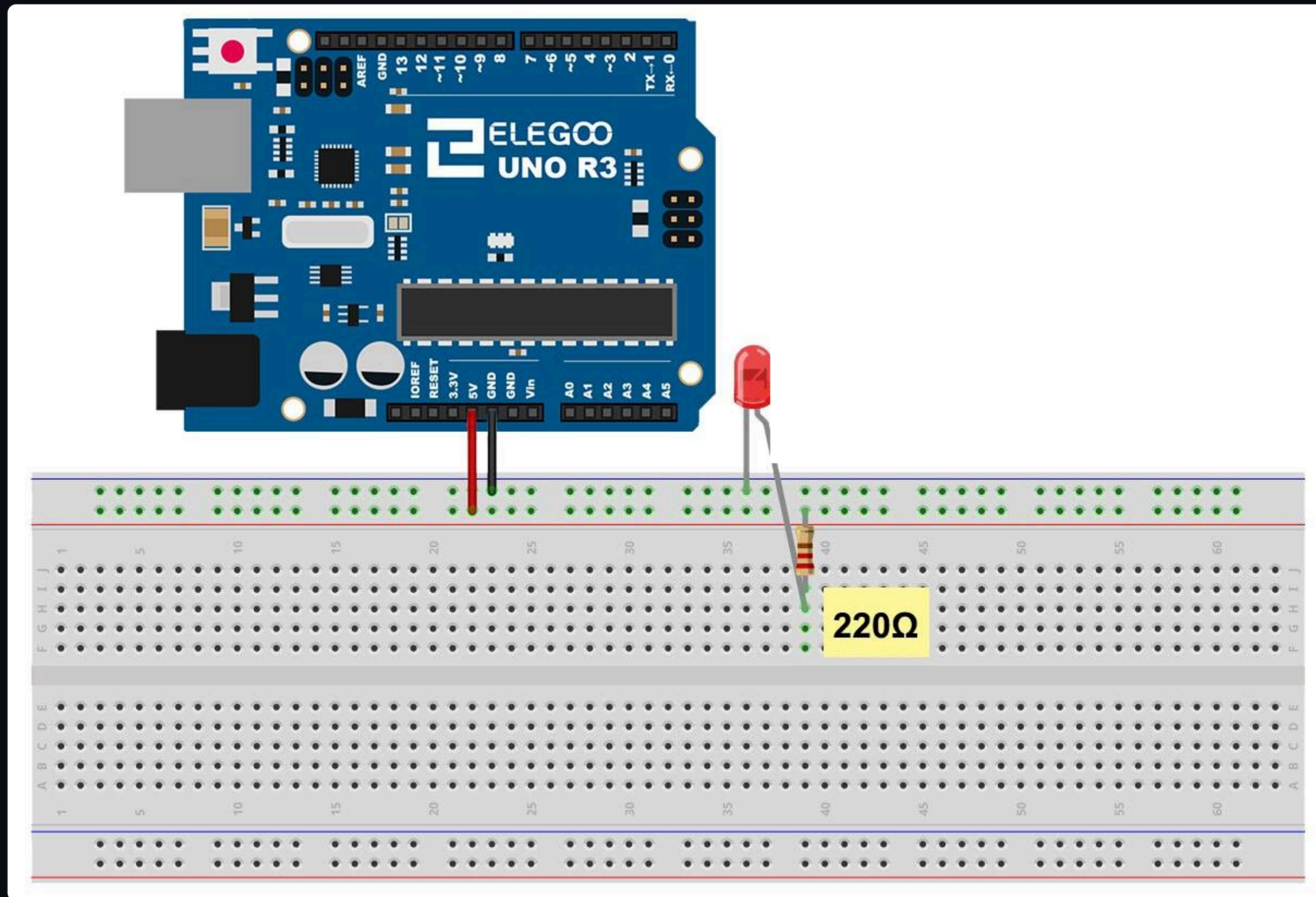
Kabeln och komponentbenet måste
sitta på **samma rad.**

PRAKTISKT MOMENT

Bygg kretsen.

- 01 Sätt LED:en på breadboarden. **Långt ben** i en ledig rad, **kort ben** i raden bredvid.
- 02 Kabel från **pin 13** på Arduinon → samma rad som långa benet.
- 03 **220 Ω -resistor** (4-band: röd-röd-brun; 5-band: röd-röd-svart-svart-brun). Ett ben i samma rad som korta LED-benet. Andra benet i valfri ledig rad.
- 04 Kabel från resistorns andra ben → **GND** på Arduinon.
- 05 Ladda upp **File** → **Examples** → **01.Basics** → **Blink**. Lampan ska börja blinka.

Så här ska det se ut.



D13 → LED långt ben · LED kort ben → 220 Ω → GND

setup() och loop() .

void setup()

Körs **en gång** när Arduinon startar eller när ni laddar upp.

Används för konfiguration: vilka pinnar är utgångar, hastighet på Serial, etc.

void loop()

Körs **om och om igen**, så länge kortet har ström. Här bor logiken: läs, reagera, vänta, upprepa.

Regel: exakt en setup() och en loop() per Arduino-program ("sketch"). Aldrig fler.

```
void setup() {  
  // Körs en gång vid uppstart.  
  pinMode(13, OUTPUT);  
}  
  
void loop() {  
  // Körs om och om igen.  
  digitalWrite(13, HIGH);  
  delay(1000);  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```


TRE KOMMANDON

Hela Blink — på tre rader.

```
pinMode(13,  
OUTPUT);
```

"Pin 13 är en **utgång**." Säger åt Arduinon att den ska skicka ut ström på den pinnen. Körs i `setup()`.

```
digitalWrite(13,  
HIGH);
```

"Sätt pin 13 till **5 V**." HIGH = tänd (5 V), LOW = släckt (0 V). Körs i `loop()`.

```
delay(1000);
```

"Vänta **1000 millisekunder**." 1000 ms = 1 sekund. Arduinon pausar allt annat under tiden.

Med bara dessa tre kan ni få en lysdiod att blinka i vilken rytm som helst.

Din egen rytm.

Ändra **Blink** till ett mönster du själv väljer.

- SOS-rytm: . . . — — — . . .
- Ditt eget tempo — snabb, långsam, oregelbunden
- Hjärtslag — två snabba, sen paus

DELRESULTAT • TRÄFF 1

KLART

Ni har byggt en blinkare.

Er första rad kod som styr den fysiska världen.

Er första krets

LED + resistor + GND

Er första sketch

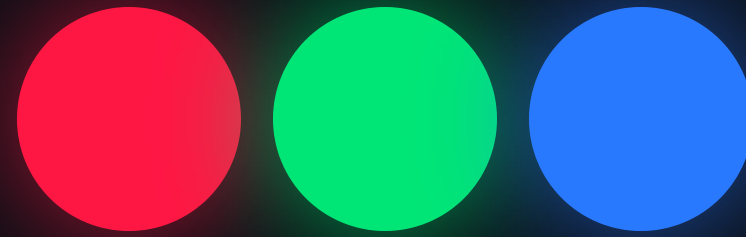
setup, loop, tre
kommandon

Er egen rytm

kod som styr fysiska
världen

Ett varv runt rummet — visa upp ditt mönster
för grannen.

NÄSTA TRÄFF



Nästa gång...

Ni kan tända en lampa i en egen
rytm.

Nästa steg: blanda **färg** ur rött,
grönt och blått.

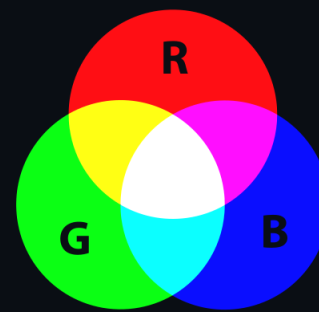
TRÄFF 2 AV 5

PWM & RGB

Modul 2: **analogWrite & färgblandning**

TRE KANALER, ALLA FÄRGER

Alla färger, av bara tre.



Skärmen ni tittar på just nu har **miljoner pixlar**, och varje pixel är bara en röd, en grön och en blå punkt. Idag programmerar vi **er egen pixel**.

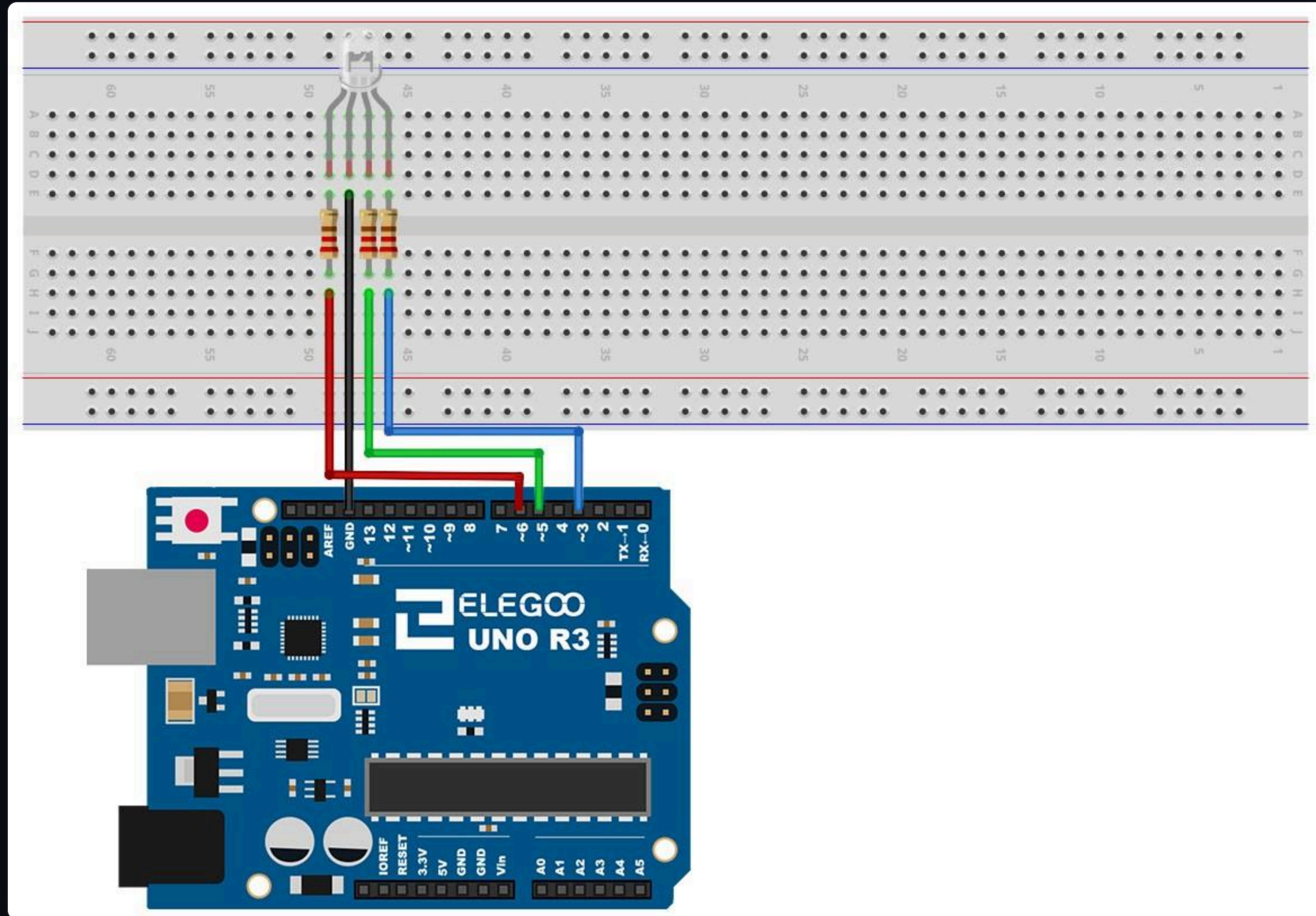
VIKTIGT — RGB-LED:EN

Common Cathode. 4 ben i rad.

Ordning från platta sidan: Röd · **Katod** · Grön · Blå

Katoden är **längst**, sitter **andra från platta sidan**, och går till **GND**.

Koppla RGB-LED:en.



R → D6 · G → D5 · B → D3 · katod → GND · varje färg via egen 220 Ω.

Gör detta först: dra en kabel från Arduinons GND till --skenan på breadboarden. Då har alla komponenter en minus-väg.

NYTT KOMMANDO • MELLAN AV OCH PÅ

`analogWrite`.

`digitalWrite` kunde bara två saker: `HIGH` eller `LOW`.

`analogWrite` tar ett tal från `0` till `255` — allt däremellan är `PWM`.

```
analogWrite(ledR, 200); // röd hög
analogWrite(ledG,  0); // grön av
analogWrite(ledB, 200); // blå hög → lila
```

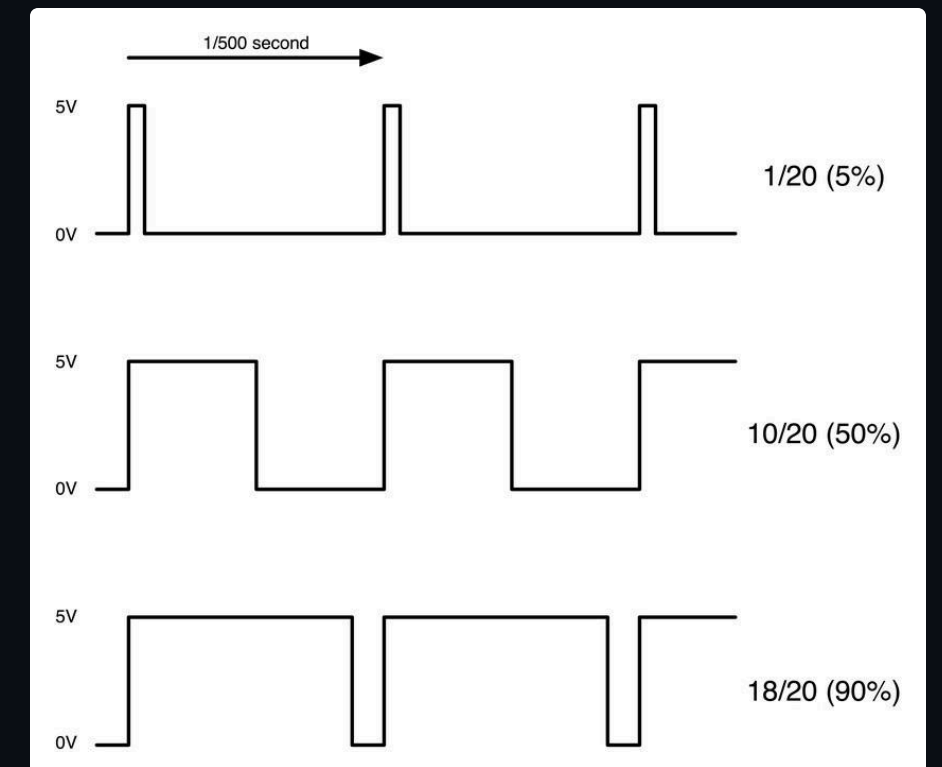
P W M

Duty cycle.

0 → alltid LOW → släckt

128 → 50 % PÅ-tid → halvstyrka

255 → alltid HIGH → full



ÖVNING

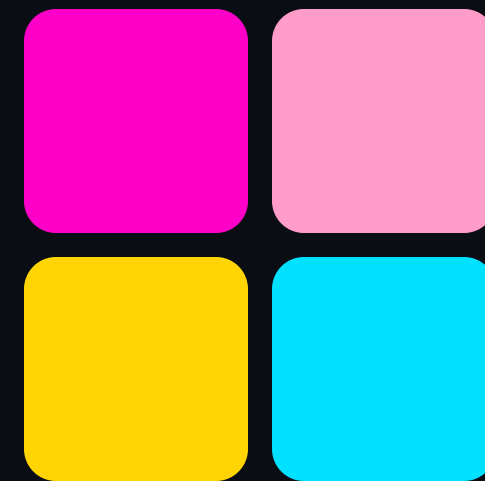
Hitta färgen.

Experimentera med värden **0–255** på varje kanal. Ladda upp, titta, justera.

- **Lila** — röd + blå, ingen grön
- **Gammelrosa** — mycket röd, lite blå, knapp grön
- **Skolgul** — full röd, lagom grön, ingen blå
- **Cyan** — ingen röd, full grön + blå

REGELN

Grön lapp = du hittade en färg du gillar. Röd lapp = du fastnat på något.



EFTER TRÄFF 2

Ni har en pixel.



Ni kan blanda *alla färger* ur tre kanaler, och ni har lärt Arduinon att leverera något *mellan* helt av och helt på.

Nästa gång: vi lyssnar. Knapp + buzzer.

TRÄFF 3 AV 5

Digital input

Modul 3: **Knapp, buzzer, logik**

NAMNGIVNA VÄRDEN

`const int` och `int`.

I morse har ni redan använt `const int` — för pin-nummer.

Nu behöver ni också `int` (utan `const`) — för värden som **ska kunna ändras under tiden programmet kör**: räknare, tillstånd, sensormätningar.

TUMREGEL

`const int` = sätts en gång, aldrig mer.

`int` = kan ändras.

`bool` = sant eller falskt.

```
const int knappPin = 9;    // ändras aldrig
int tryckCount = 0;        // kan öka
bool larmPasLaget = false; // kan togglas
```

Full förklaring av datatyper, scope och operatorer i kompendiets Appendix A.

D I G I T A L I N P U T

Input ≠ Output.

Hittills har Arduinon bara skickat ström ut
(OUTPUT).

Nu läser den av ett tillstånd **in** (INPUT).

```
pinMode(knappPin, INPUT_PULLUP); // intern pullup-resistor  
int niva = digitalRead(knappPin); // LOW = tryckt, HIGH = släppt
```

VARFÖR PULLUP?

En pinne som bara "lyssnar" utan koppling till plus eller minus *flyter* — läsningen blir slumpmässig.

INPUT_PULLUP låter Arduinon själv dra pinnen till HIGH. När du trycker dras den till GND (LOW).

Därför: tryckt = LOW, släppt = HIGH.

FATTA BESLUT

`if` / `else` .

Läs ett värde → jämför → agera olika beroende på resultatet.

Operatorerna: `==` lika, `!=` olika, `<` mindre, `>` större.

VANLIGASTE NYBÖRJARFELET

= tilldelar värde.

`==` jämför.

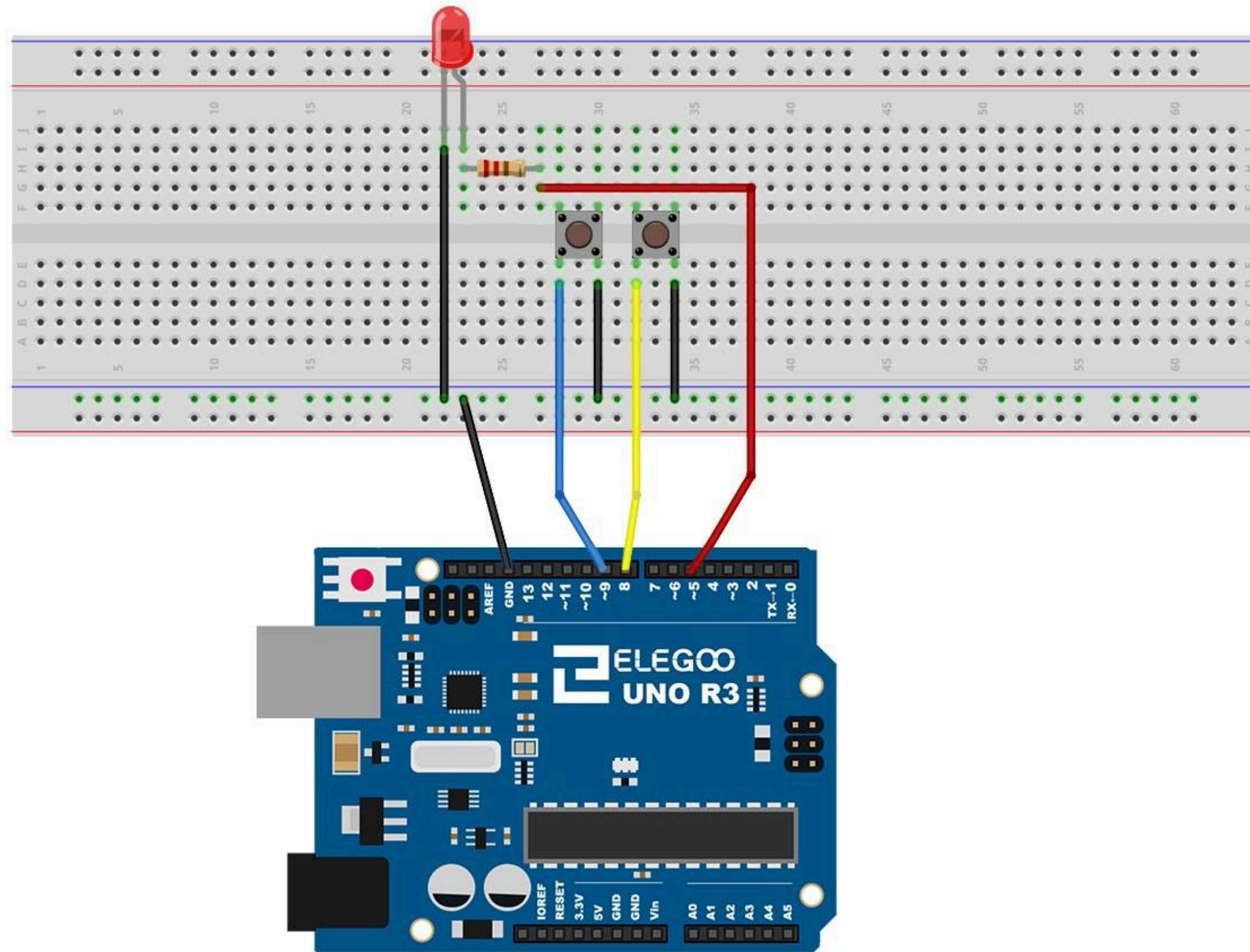
Glöm inte andra likhetstecknet.

```
if (digitalRead(knappPin) == LOW) {  
  // knappen är tryckt  
  digitalWrite(LED_BUILTIN, HIGH);  
} else {  
  // släppt  
  digitalWrite(LED_BUILTIN, LOW);  
}
```

Fler operatorer (&&, ||, !)

och kedjade if/else i kompendiets Appendix A.

Koppla knappen.



Vi använder bara knapp A → D9 · andra benet till GND. Ingen pulldown-resistor — `INPUT_PULLUP` sköter det internt.

Reagera på flanken.

En knapp som **hålls nere** skulle toggla 50 gånger per sekund.
Resultat: larmet blinkar mellan av/på som ett stroboskop.

Lösningen: agera inte på att knappen *är* nere. Agera på att den **just nu gick från HIGH till LOW** — flanken.

MÖNSTRET

Spara förra värdet. Jämför med nuvarande. Om det ändrats åt rätt håll → det är en flank → agera en gång.

```
bool larmPasLaget = false;
int lastState = HIGH;

void loop() {
  int state = digitalRead(knappPin);
  if (state == LOW && lastState == HIGH) {
    // just nu tryckt - toggla
    larmPasLaget = !larmPasLaget;
  }
  lastState = state;
  delay(10); // mot studs
}
```

ACTIVE BUZZER

En varning först.

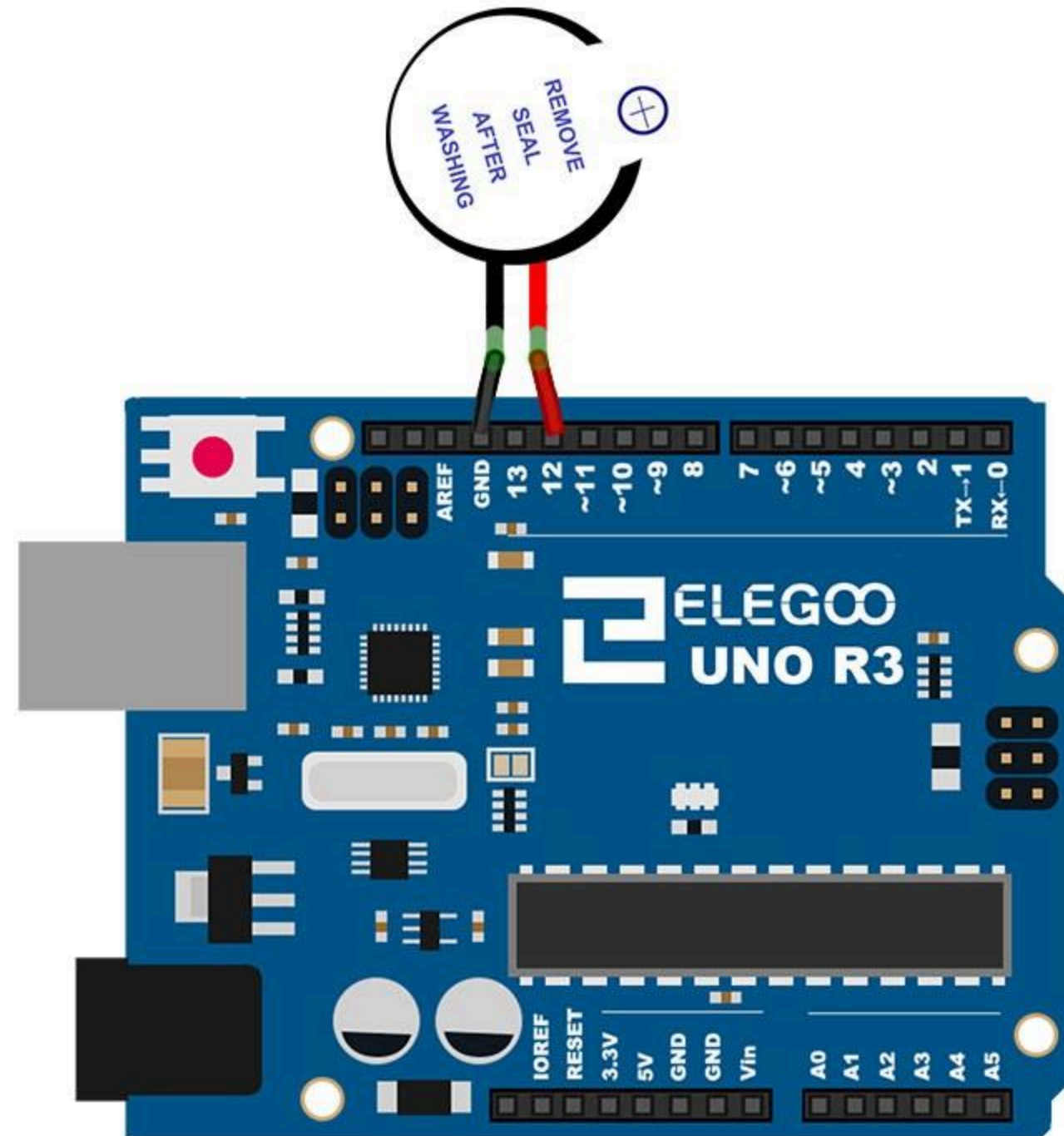
❗ EXTREMT VIKTIGT

Dra absolut inte av den lilla klisterlappen
"Remove after washing"
på er buzzer.

Gör ni det kommer era öron att **blöda** och jag kastar ut er. Den är vår ljuddämpare.



Koppla buzzern.



Långt ben (+) → D12 · kort ben → GND. Rakt in i Arduino-headerna. Ingen breadboard, ingen resistor.

SÄTT IHOP ALLT

Läs knapp. Styr buzzer.

Knappen ner → buzzern tjuder.

Släpp → tyst.

INPUT → `if / else` → OUTPUT

SÄTT IHOP ALLT • KÄRNAN I LOOP()

Kärnan i `loop()`.

```
if (digitalRead(knappPin) == LOW) {  
    digitalWrite(buzzerPin, HIGH);  
} else {  
    digitalWrite(buzzerPin, LOW);  
}
```

Active buzzer: HIGH = pip, LOW = tyst. Ingen tone() behövs.

EFTER TRÄFF 3

Arduinon lyssnar.

Ni har byggt Arduinons första **sinnesorgan** —
en knapp — och gett den en **röst** — buzzern.
En läser, den andra reagerar.

*Nästa gång: **analoga sensorer**. Mörker, rörelse, mätvärden. Serial Monitor.*

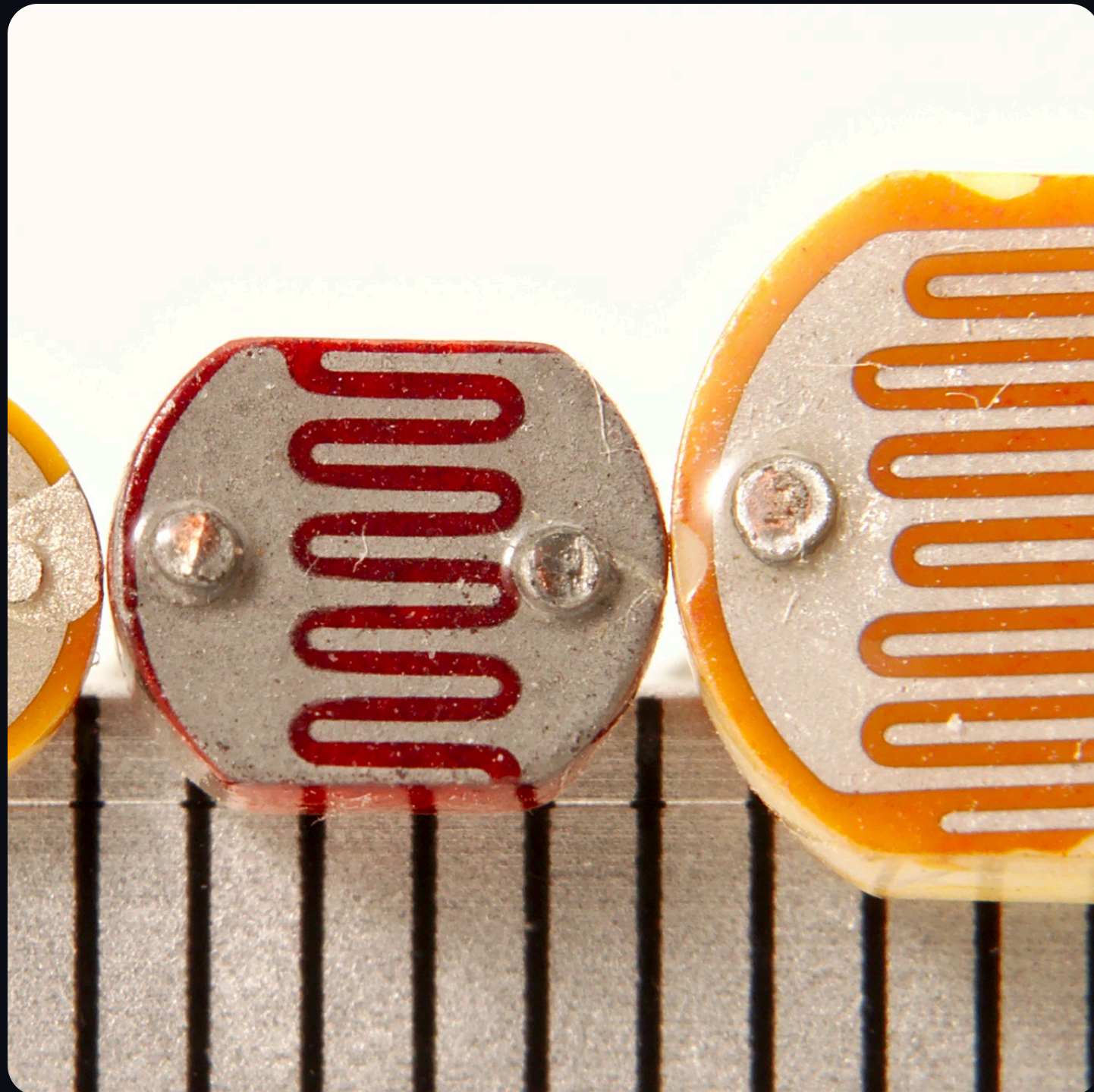
TRÄFF 4 AV 5

Analog input

Modul 4: **Sensorer & Serial Monitor**

ANALOG INPUT

Knappen var digital. Världen är analog.



Knappen: PÅ eller AV.

Ljuset: halvmörkt, starkt, svagt.

Fotocell (LDR) mäter ljus: 0-1023

NYTT KOMMANDO

`analogRead()` — Arduinos linjal.

```
int ljus = analogRead(A0); // 0 - 1023
```

GÖR DETTA NU

Bygg spänningsdelaren: fotocell + 1 k Ω →
pin `A0`

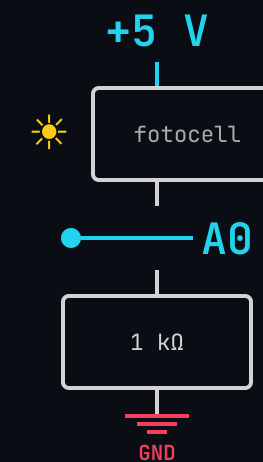
NYTT KONCEPT · FRÅN MOTSTÅND TILL MÄTVÄRDE

Spänningsdelaren.

Tänk dig ett vattenrör som höjdskala: `+5 V` är uppe, `GND` är nere. Två motstånd i serie är två rörsegment; skarven mellan dem är en tapp — där mäter `A0`.

- Lika stora → skarven mitt på → `A0 ≈ 2,5 V`.
- Mer motstånd **ovanför** → skarven pressas ner → `A0` sjunker.
- Mer motstånd **nedanför** → skarven dras upp → `A0` stiger.

Fotocellen är det övre motståndet. Mörker → högt → `A0` sjunker. Ljus → lågt → `A0` stiger. `analogRead(A0)` ger 0–1023.



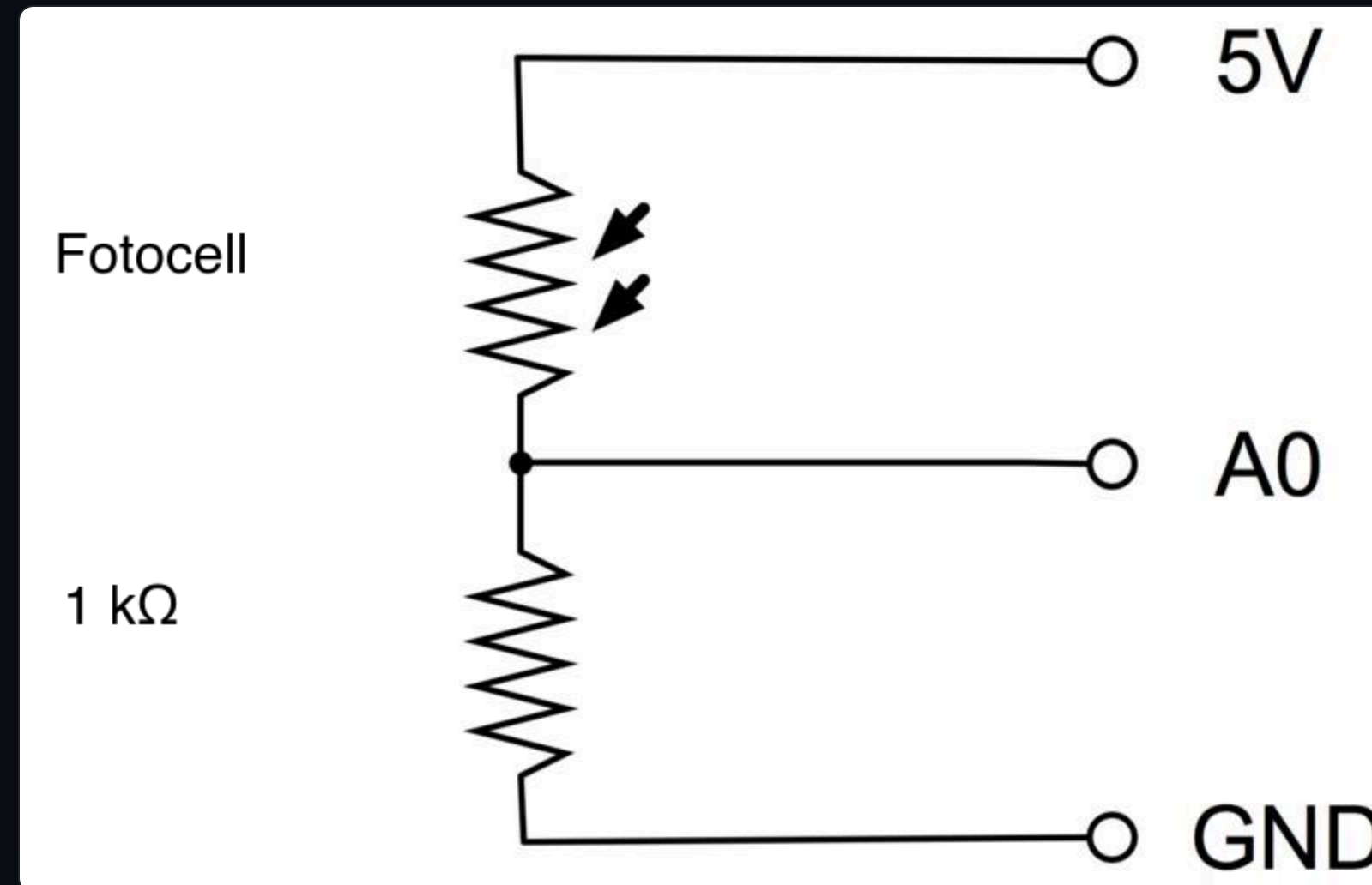
TYPISKA VÄRDEN · 1 kΩ

Hand över → 20–100

Rumsljus → 150–400

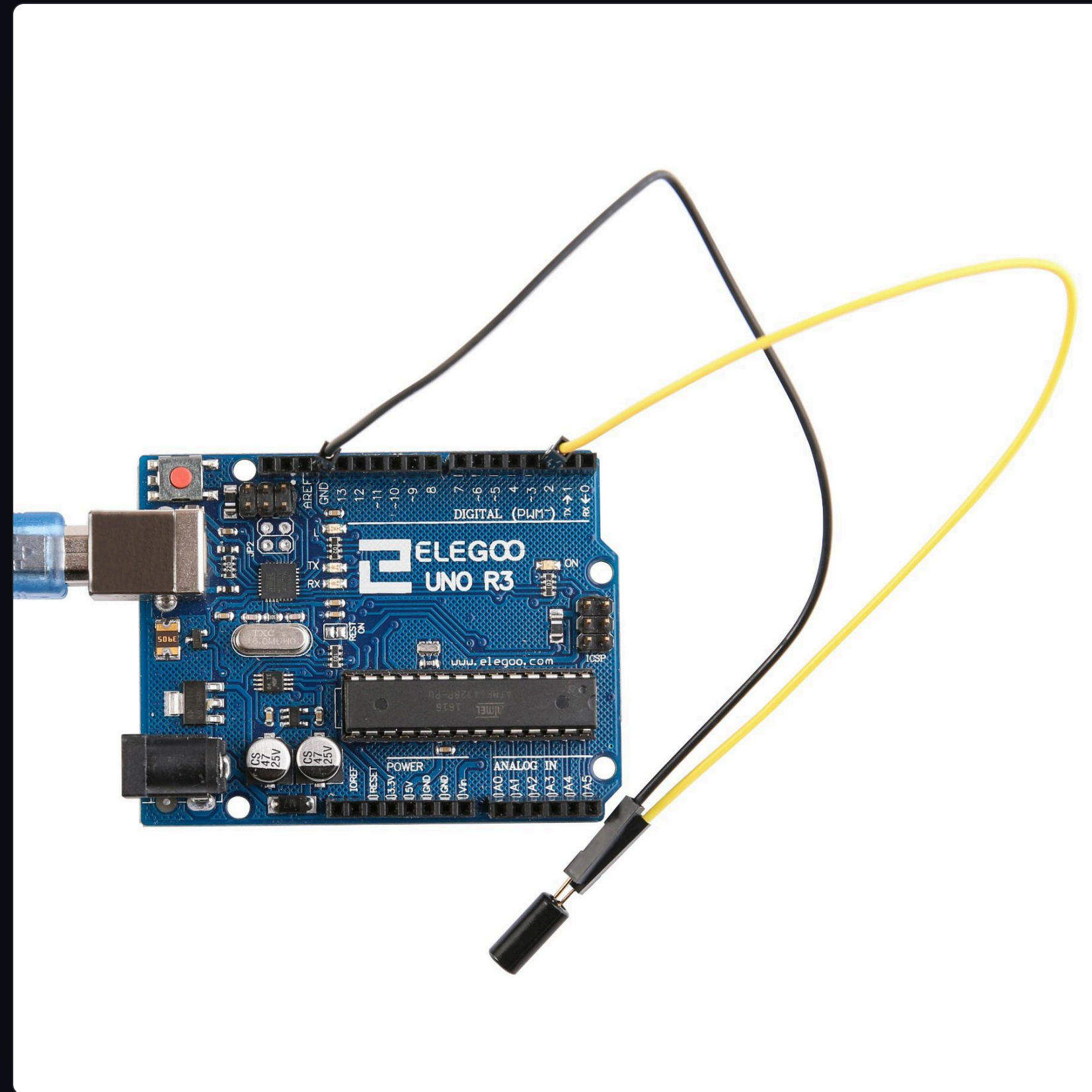
Lampa nära → 500–700

Koppla fotocellen.



Spänningsdelare: 5 V → fotocell → A0 → 1 kΩ → GND. Mörker ger lågt värde, ljus ger högt värde.

Koppla tilt-sensorn.



Rakt in i Arduino-headerna med F-M DuPont-kablar: **ena benet → D2**, **andra benet → GND**. Kräver ingen breadboard. Digital sensor — öppen/sluten, precis som en knapp. Läsas med `digitalRead`, inte `analogRead`.

F E L S Ö K N I N G

Serial Monitor.

Hur ser vi vilka värden Arduinon **läser**?

Vi låter den skriva ut dem till datorn över **USB**.

Öppna **förstöringsglaset** uppe till höger i Arduino IDE.

```
void setup() {  
  Serial.begin(9600);  
}  
  
void loop() {  
  int ljus = analogRead(A0);  
  Serial.println(ljus);  
  delay(100);  
}
```

Hitta din tröskel.

Printa **både** ljus och tilt till Serial Monitor. Jämför siffrorna mot omgivningen.

- Täck fotocellen — vilket värde?
- Lys på den — vilket värde?
- Håll tilt-sensorn upprätt, sen lutad — HIGH → LOW?

Koden att köra.

```
void loop() {  
    int ljus = analogRead(A0);  
    int tilt = digitalRead(2);  
    Serial.print("ljus=");  
    Serial.print(ljus);  
    Serial.print(" tilt=");  
    Serial.println(tilt);  
    delay(200);  
}
```

Full sketch med `setup()` i kompendiet, kapitel 4.

EFTER TRÄFF 4

Arduinon känner världen.

Ni kan läsa **ljus** med en fotocell, **lutning** med en tilt-sensor, och **titta in** i Arduinons hjärna via Serial Monitor.

*Nästa vecka: **hackathon**. Ni sätter ihop allt till ett fungerande tjuvlarm. Kom hungriga.*

TRÄFF 5 AV 5

Integration

Modul 5: **Hackathon — bygg larmet**

HACKATHON

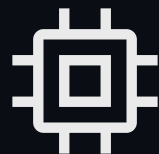
Ingenjörssuppgiften.

Ingen manual att kopiera. Ni har alla delar. **Sätt ihop dem.**



INPUT

Knapp · LDR · Tilt



LOGIK

Arduino & kod



OUTPUT

RGB-LED · Buzzer

Regler: Knappen togglar larm-läge. Larm PÅ + tilt rörd = tjut + rött blink.
Larm AV + mörkt = stämningsljus.

Pin-tilldelning

Samma pinnar som ni använt i varje lektion.

VARIABEL	PIN	MODUL
knappPin	D9	Modul 3
tiltPin	D2	Modul 4
ldrPin	A0	Modul 4
buzzerPin	D12	Modul 3
ledR / G / B	6 / 5 / 3	Modul 2

```
bool larmPaslaget = false;
```

→ Kompletta startmall med `setup()` och `loop()`: Bilaga D

EFTER FEM TRÄFFAR

Tack.

Ni har byggt ett system som

läser av omvärlden

och reagerar på den.

Det är grunden i inbyggda

system.

FR0 Ånge • 2026